Práctica: Algoritmo genético para el problema de flujo de permutación (PFSP)

Inteligencia Artificial — 3.º Grado en Ingeniería Informática

Duración: 2 semanas • Entrega: 19-23 de octubre

1. Objetivos

Al terminar esta práctica deberás ser capaz de:

- Modelar un problema de secuenciación y diseñar una representación adecuada de sus soluciones.
- Implementar un algoritmo genético (AG) completo: inicialización, selección, cruce, mutación y reemplazo.
- Diseñar y justificar operadores que generen *siempre* soluciones factibles.
- Evaluar experimentalmente un algoritmo estocástico y comparar sus resultados con los de la literatura usando instancias de referencia.

2. Permutation Flow Shop Scheduling Problem (PFSP)

2.1. Definición

Se dispone de n trabajos $J_1, \ldots, J_n$ que deben procesarse en m máquinas $M_1, \ldots, M_m$. Todos los trabajos recorren las máquinas en el mismo orden (M_1, después M_2, …, hasta M_m), y p_{ij} es el tiempo de proceso del trabajo J_j en la máquina M_i. Se asume que:

- cada máquina procesa como mucho un trabajo a la vez y cada trabajo está como mucho en una máquina a la vez;
- una vez empezada, una operación no se interrumpe;
- todos los trabajos están disponibles en el instante 0;
- el orden en que se procesan los trabajos es el **mismo en todas las máquinas** (de ahí "de permutación").

Una solución es, por tanto, una permutación $\pi = (\pi_1, \ldots, \pi_n)$ de los trabajos. El objetivo es minimizar el **makespan** $C_{\max}(\pi)$, es decir, el instante en que termina el último trabajo en la última máquina.

2.2. Cálculo del makespan

Sea $C(k, i)$ el instante en que el trabajo situado en la posición k termina en la máquina i. Un trabajo no puede empezar en la máquina i hasta que esa máquina haya terminado con el trabajo anterior y hasta que el propio trabajo haya salido de la máquina $i-1$:

$$
\begin{aligned}
C(k,i) &= \max\{C(k-1,i),\ C(k,i-1)\} + p_{i\pi_k}, \qquad k = 1,\ldots,n,\ i = 1,\ldots,m, \qquad (1)\\
C(0,i) &= 0, \quad C(k,0) = 0,\\
C_{\max}(\pi) &= C(n,m).
\end{aligned}
$$

El cálculo cuesta $O(nm)$. El problema es NP-duro para $m \geq 3$.

2.3. Ejemplo

Considera la instancia con $n = 4$ trabajos y $m = 3$ máquinas del cuadro 1.

p_{ij}	J_1	J_2	J_3	J_4
M_1	6	4	2	7
M_2	1	3	7	7
M_3	1	1	7	1

Tabla 1: Tiempos de proceso de la instancia de ejemplo ($n = 4$, $m = 3$).

La figura 1 muestra los diagramas de Gantt de dos secuencias. Con $\pi = (1, 2, 3, 4)$ se obtiene $C_{\text{máx}} = 28$: el trabajo J_1, largo en M_1, retrasa a todos los demás, y J_3 y J_4 se atascan en M_2 y M_3. Con $\pi = (3, 4, 2, 1)$ se obtiene $C_{\text{máx}} = 21$, que es el óptimo de esta instancia (de las 24 secuencias posibles, es la única que lo alcanza).

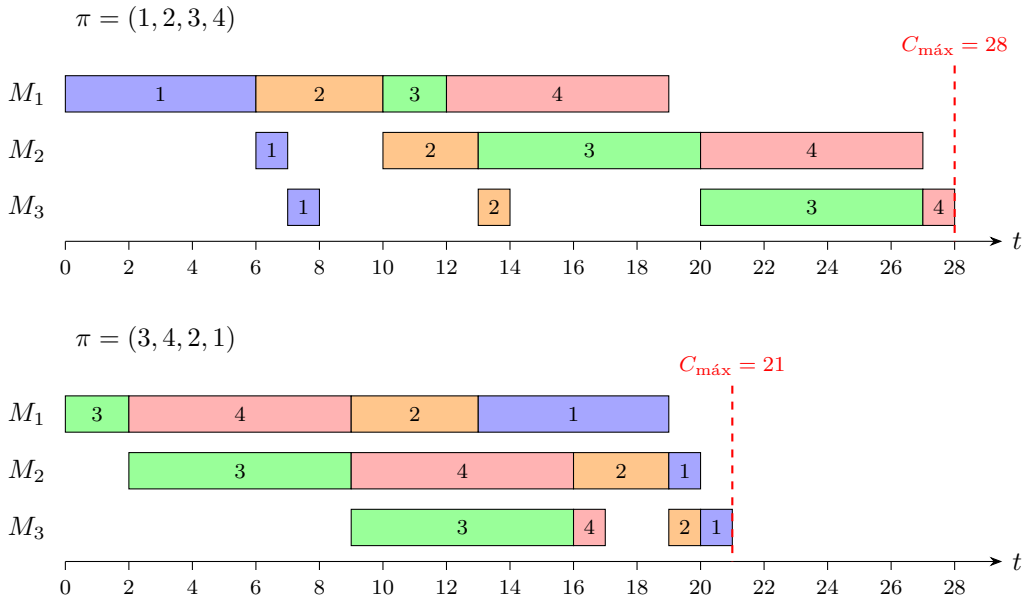


Figura 1: Diagramas de Gantt de la instancia de ejemplo para dos secuencias. Cada rectángulo es la operación de un trabajo (número) en una máquina. La secuencia $(3, 4, 2, 1)$ es óptima.

Comprueba a mano algunos valores con la recurrencia (1). Por ejemplo, en la secuencia $(1, 2, 3, 4)$, el trabajo J_3 termina en M_1 en el instante 12, pero no puede entrar en M_2 hasta el 13, cuando M_2 queda libre de J_2: $C(3, 2) = \text{máx}\{13, 12\} + 7 = 20$.

3. Representación de las soluciones

Una solución se representa mediante una **permutación** π de los n trabajos: π_k es el trabajo que se procesa en la posición k (en todas las máquinas). Por ejemplo, para $n = 8$:

posición	1	2	3	4	5	6	7	8
$\pi =$	3	7	1	5	2	8	4	6

Figura 2: Representación de una solución como permutación: el trabajo 3 se procesa primero, el 7 en segundo lugar, etc.

Esta representación tiene dos propiedades importantes:

1. **Toda permutación es una solución factible**: la recurrencia (1) construye una planificación válida para cualquier orden de los trabajos.
2. El espacio de búsqueda tiene $n!$ soluciones; para $n = 20$ son más de $2 \cdot 10^{18}$, por lo que la enumeración exhaustiva es inviable incluso en las instancias pequeñas.

La dificultad está en los operadores: el cruce de un punto clásico de los AG binarios *no* sirve, porque produce trabajos repetidos y trabajos ausentes.

4. El algoritmo genético

El esquema general que debes implementar es el del algoritmo 1.

Algoritmo 1 Esquema del algoritmo genético para el PFSP

Entrada: Tiempos p_{ij}, tamaño de población N, probabilidades p_c, p_m, presupuesto de evaluaciones E
Salida: Mejor secuencia encontrada π^*

$P \leftarrow$ InicializarPoblación(N) ▷ aleatoria
Evaluar cada individuo de P; $\pi^* \leftarrow$ mejor de P
while número de evaluaciones $< E$ **do**
  $P' \leftarrow \emptyset$
  while $|P'| < N$ **do**
    $p_1, p_2 \leftarrow$ Selección(P) ▷ p. ej., torneo binario
    Con probabilidad p_c: $h_1, h_2 \leftarrow$ CruceOX(p_1, p_2); si no, $h_1, h_2 \leftarrow p_1, p_2$
    Con probabilidad p_m aplicar Mutación a h_1 y a h_2
    Evaluar h_1, h_2 y añadirlos a P'
  end while
  $P \leftarrow$ Reemplazo(P, P') ▷ p. ej., generacional con elitismo
  Actualizar π^*
end while
return π^*

4.1. Población inicial y la heurística NEH

Además de permutaciones aleatorias, puedes incluir una solución construida con la heurística **NEH** (Nawaz, Enscore y Ham, 1983), considerada la mejor heurística constructiva para este problema:

1. Ordenar los trabajos de mayor a menor tiempo total de proceso $\sum_i p_{ij}$.
2. Tomar los dos primeros y quedarse con el mejor de sus dos órdenes posibles.
3. Para cada trabajo restante, en el orden del paso 1, probar a insertarlo en todas las posiciones de la secuencia parcial y quedarse con la que dé menor makespan.

Cuida la diversidad: una población formada solo por variaciones de la solución NEH converge prematuramente.

4.2. Selección

Se recomienda el **torneo** de tamaño k (con $k = 2$ como valor inicial): se eligen k individuos al azar y se selecciona el de *menor* makespan.

4.3. Cruce de orden (OX)

El cruce de orden (*Order Crossover*, OX) fue propuesto por Davis (1985) para problemas de secuenciación. Garantiza que los hijos son siempre permutaciones válidas y, por tanto, soluciones factibles. Para obtener el hijo h_1 a partir de los padres p_1 y p_2:

1. Elegir al azar dos puntos de corte $a \leq b$.
2. Copiar en h_1 el segmento $p_1[a..b]$, en las mismas posiciones.
3. Recorrer p_2 de izquierda a derecha, saltando los trabajos que ya están en h_1, y colocar los restantes, en ese orden, en las posiciones libres de h_1 de izquierda a derecha.

El hijo h_2 se obtiene intercambiando los papeles de los padres. La figura 3 muestra un ejemplo con $n = 8$ y segmento en las posiciones 3 a 5.

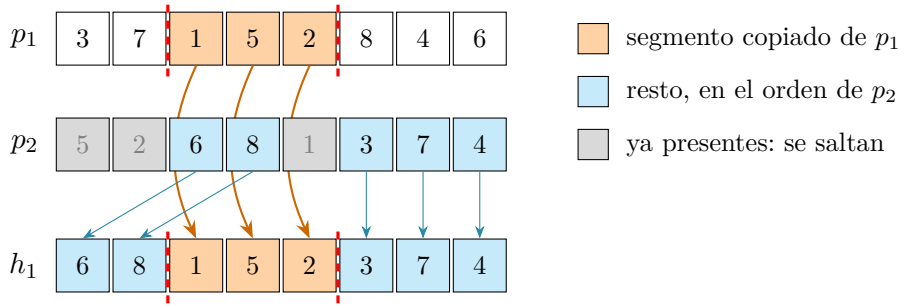


Figura 3: Cruce OX. Se copia el segmento $(1, 5, 2)$ de p_1; los trabajos que faltan se toman de p_2 en su orden de aparición $(6, 8, 3, 7, 4)$ y rellenan los huecos de izquierda a derecha. Resultado: $h_1 = (6, 8, 1, 5, 2, 3, 7, 4)$.

Con los mismos puntos de corte, el segundo hijo copia el segmento $(6, 8, 1)$ de p_2 y completa con $(3, 7, 5, 2, 4)$ en el orden de p_1: $h_2 = (3, 7, 6, 8, 1, 5, 2, 4)$.

¿Por qué OX es adecuado para el PFSP? En el PFSP importan tanto el *orden relativo* de los trabajos como los *bloques* de trabajos consecutivos que encajan bien entre sí (como J_3, J_4 en el ejemplo, que se solapan sin dejar huecos en las máquinas). OX conserva intacto un bloque de p_1, en sus mismas posiciones, y mantiene el orden relativo del resto de trabajos según p_2.

> **Nota.** En la versión original de Davis el relleno empieza en la posición siguiente al segundo corte y continúa de forma circular. La versión descrita aquí (relleno de izquierda a derecha) también se usa con frecuencia. Puedes implementar cualquiera de las dos, pero debes indicar cuál y justificar la elección.

4.4. Mutación

Se proponen dos operadores, ambos cerrados sobre permutaciones:

- **Intercambio** (*swap*): se intercambian los trabajos de dos posiciones aleatorias.
- **Inserción** (*shift*): se extrae el trabajo de una posición i y se inserta en otra posición j, desplazando los intermedios. En el PFSP suele funcionar mejor que el intercambio.

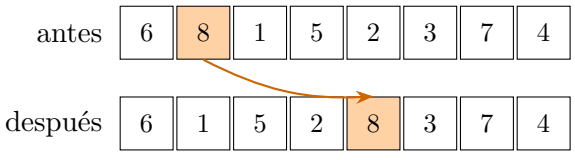


Figura 4: Mutación por inserción: el trabajo 8 se mueve de la posición 2 a la 5; los trabajos 1, 5 y 2 se desplazan una posición a la izquierda.

4.5. Reemplazo

Como punto de partida usa un **reemplazo generacional con elitismo**: la nueva población son los hijos, pero el mejor individuo de la generación anterior sustituye al peor hijo si es mejor

que él. Puedes comparar con un esquema estacionario, en el que cada pareja de hijos compite con los peores de la población.

5. Instancias y resultados de referencia

Se usarán las instancias de **Taillard** (1993), el conjunto de referencia estándar para este problema: 120 instancias en 12 grupos de 10, con tamaños $n \times m$ desde 20×5 hasta 500×20.

En el aula virtual se facilitará el conjunto completo de instancias.

Formato de los ficheros: la primera línea contiene n y m, y a continuación vienen m líneas (una por máquina) con los n tiempos de proceso.

Para medir la calidad de una solución usarás la **desviación porcentual relativa** respecto a la mejor cota superior conocida (recuerda que el problema es de minimización):

$$\mathrm{RPD}(\pi) = 100 \cdot \frac{C_{\max}(\pi) - \mathrm{UB}}{\mathrm{UB}}.$$

Como referencia, la heurística NEH obtiene desviaciones medias en torno al 3 % sobre estas instancias, y los AG híbridos de Ruiz, Maroto y Alcaraz (2006) bajan del 1 %.

6. Tareas

Puedes usar el lenguaje que prefieras (C++, Java, Python…).

T1. Lectura y evaluación. Implementa la lectura de instancias y el cálculo del makespan con la recurrencia (1). Comprueba que con la instancia del cuadro 1 obtienes $C_{\max}(1,2,3,4) = 28$ y $C_{\max}(3,4,2,1) = 21$.

T2. Algoritmos de referencia. Implementa:

(a) una *búsqueda aleatoria* que genere y evalúe permutaciones aleatorias hasta agotar el presupuesto, y
(b) la *heurística NEH*.

Tu AG debe superar claramente a ambos.

T3. Algoritmo genético. Implementa el AG del algoritmo 1 con selección por torneo, cruce OX, mutación por inserción e intercambio, y reemplazo generacional con elitismo. Todos los parámetros (N, p_c, p_m, tamaño del torneo, semilla) deben poder configurarse sin recompilar.

T4. Comprobación de factibilidad. Añade una función que verifique que cada hijo es una permutación válida y actívala durante el desarrollo.

T5. Ajuste de parámetros. Estudia al menos dos valores de N y dos de p_m sobre un subconjunto pequeño de instancias de ajuste, distinto del de evaluación. Justifica la configuración final.

T6. Experimentación. Con la configuración final, realiza 10 ejecuciones independientes (semillas distintas) por instancia y algoritmo, con un presupuesto de 50 000 evaluaciones del makespan por ejecución. Presenta:

(a) una tabla con el RPD medio por grupo de instancias ($n \times m$) y algoritmo;
(b) gráficas de convergencia (mejor makespan frente a número de evaluaciones) de al menos dos instancias;

T7. Análisis. Discute los resultados: ¿qué componente influye más? ¿Hay convergencia prematura? ¿Cuánto mejora el AG a NEH? ¿Cómo se compara con los resultados publicados?

7. Entrega y evaluación

Se entregará, a través del aula virtual la semana del 19-23 de octubre, un fichero comprimido con:

- El **código fuente**, con instrucciones de compilación y ejecución (`README`).
- Los **ficheros de resultados** brutos de todas las ejecuciones.

Tras la entrega habrá que realizar una modificación, así como una **defensa oral** de la práctica entregada. En ella se pedirá explicar el código y contestar a las preguntas realizadas por el profesor.

Criterio	**Peso**
Implementación correcta y eficiente y respuestas correctas:	35 %
Experimentación, análisis y modificación (T6–T8)	40 %
Defensa oral	25 %

Uso de herramientas de IA generativa. Puedes usar asistentes de IA como apoyo, pero recuerda que eres responsable de todo el código entregado. Si en la defensa no puedes explicarlo o modificarlo, la práctica se considerará no superada.

Referencias

[1] L. Davis. Applying adaptive algorithms to epistatic domains. *Proc. IJCAI*, 162–164, 1985.

[2] D. E. Goldberg y R. Lingle. Alleles, loci, and the traveling salesman problem. *Proc. 1st Int. Conf. on Genetic Algorithms*, 154–159, 1985.

[3] S. M. Johnson. Optimal two- and three-stage production schedules with setup times included. *Naval Research Logistics Quarterly*, 1(1):61–68, 1954.

[4] M. Nawaz, E. E. Enscore y I. Ham. A heuristic algorithm for the m-machine, n-job flow-shop sequencing problem. *Omega*, 11(1):91–95, 1983.

[5] C. R. Reeves. A genetic algorithm for flowshop sequencing. *Computers & Operations Research*, 22(1):5–13, 1995.

[6] R. Ruiz y C. Maroto. A comprehensive review and evaluation of permutation flowshop heuristics. *European Journal of Operational Research*, 165(2):479–494, 2005.

[7] R. Ruiz, C. Maroto y J. Alcaraz. Two new robust genetic algorithms for the flowshop scheduling problem. *Omega*, 34(5):461–476, 2006.

[8] E. Taillard. Benchmarks for basic scheduling problems. *European Journal of Operational Research*, 64(2):278–285, 1993.